

CS & IT ENGINEERING

Programming in C

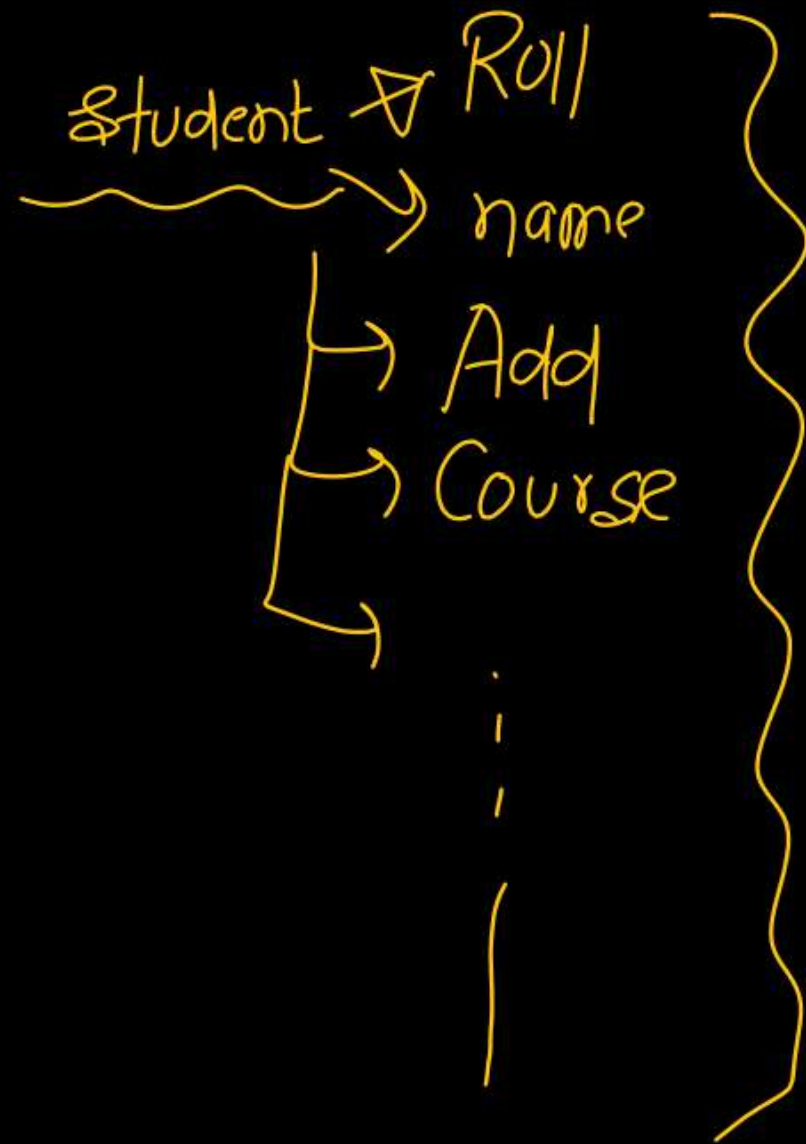
Structure and Union
Lec- 01



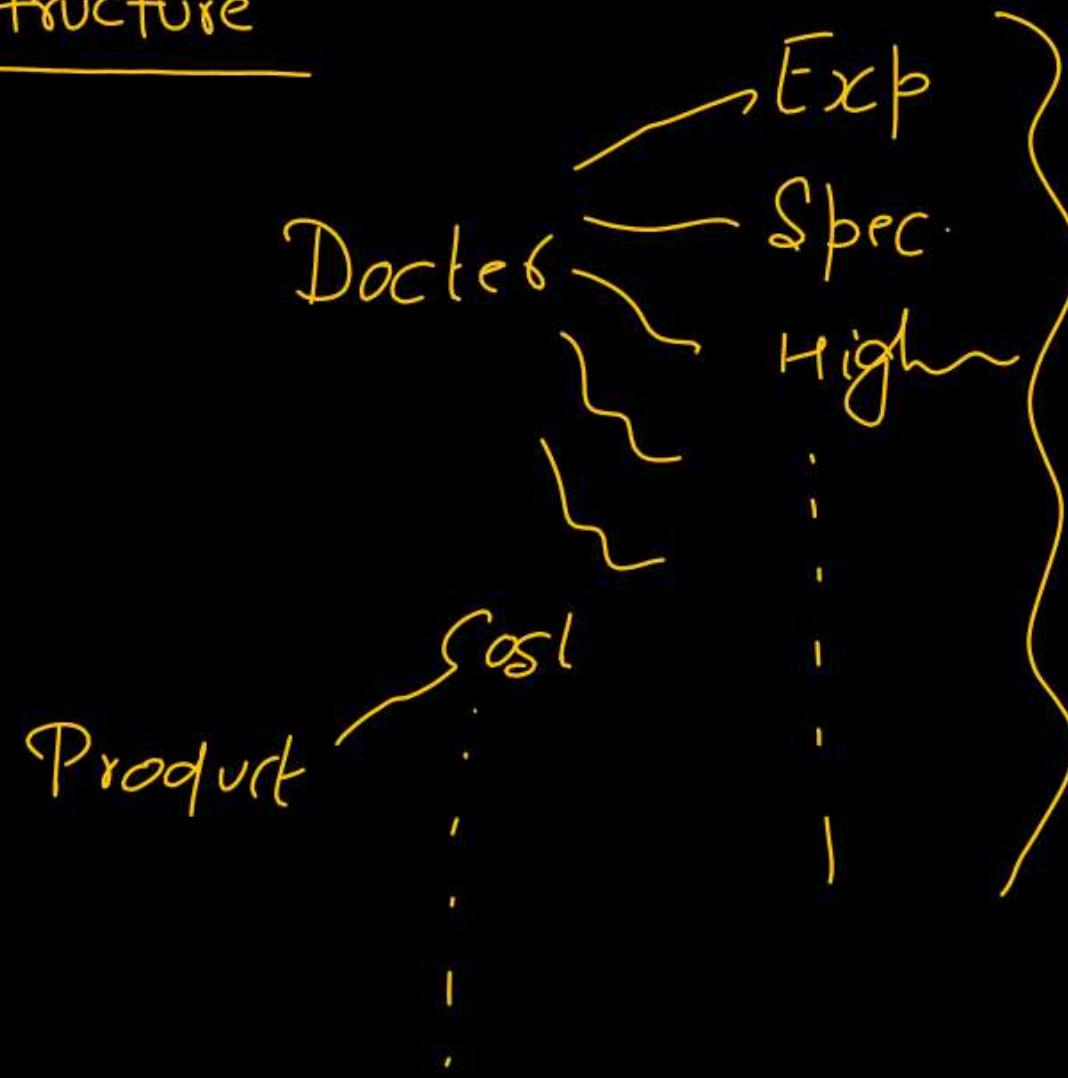
By- Pankaj Sharma sir

TOPICS TO BE
COVERED

Structure and Union-I



Structure



① int Roll ;
② char name[20] ;
③ Char Add[30] ;

Collectively

Collection of diff. types
of data element

user defined data type

User defined data type

✓
Compiler → int, float, double,
char
(Primitive)

Student → Roll
 → name

(i) struct is the keyword used to create user defined d.t.

↑
info to the
Compiler
↓

struct student {
 int Roll;
 char name[20];
};

```
struct student {  
    int Roll;  
    char name[20];  
};
```

↑
template / blueprint
↓

→ No memory
is allocated

```
void main() {  
    int ;  
}
```

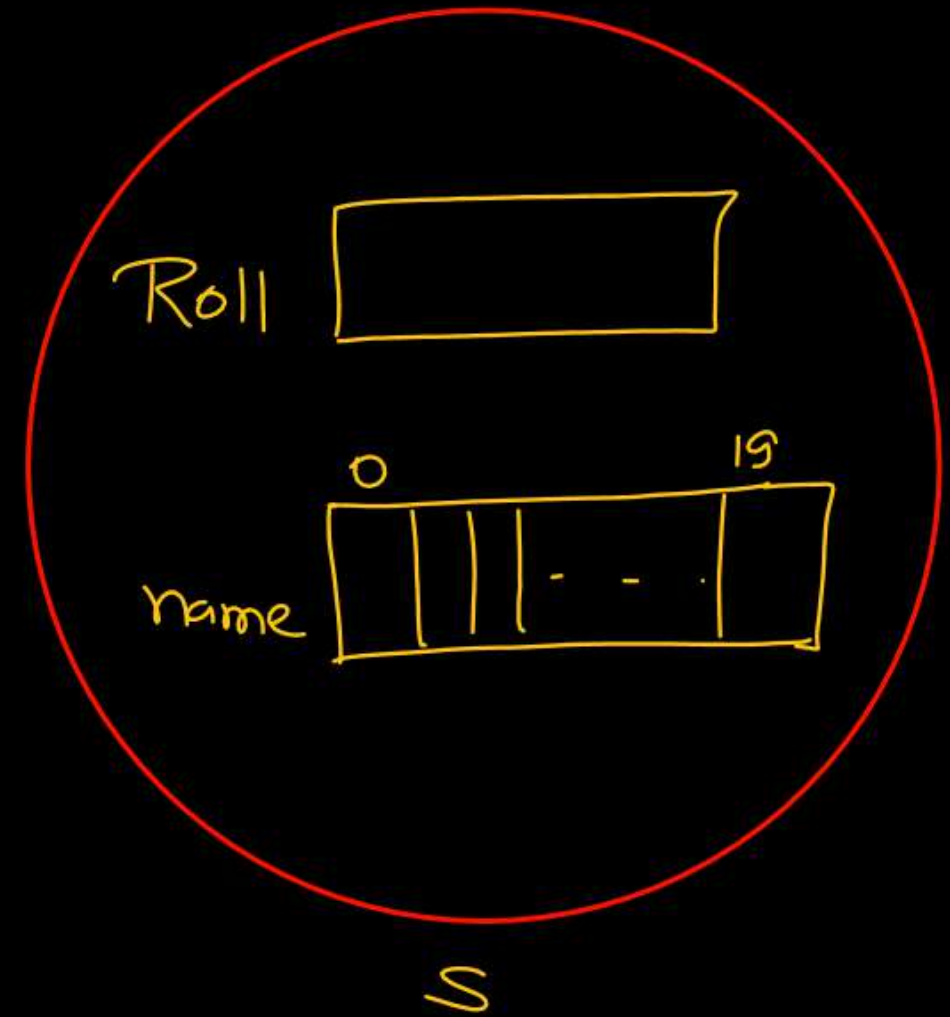
}

```
struct student {  
    int Roll;  
    char name[20];  
};
```

```
void main(){  
    struct student
```

```
    int (a,b);
```

variable
(group of
2 members)




```
struct student{
```

```
int Roll;
```

```
char name[20];
```

```
};
```

```
void main(){
```

```
struct student s1, s2;
```

```
s1.Roll = 10; ✓
```

```
s2.Roll = 20; ✓
```

```
s1.name = "Pankaj";
```

Invalid

Lvalue

```
strcpy(s1.name, "Pankaj");
```

s1.mem1

membership of

Roll 10

name

s1

Roll 20

name

s2

solution

→ Name/Tag

```
struct student {  
    int Roll;  
    char name[20];  
};
```

```
void main() {  
    struct student s;  
    ==  
}
```

```
void A() {  
    struct student s2;  
    ==  
}
```

```
struct {  
    int Roll;  
    char name[20];  
} s1, s2, s3;
```

```
void main() {  
    struct P  
}
```

```
void A() {  
}
```

```
typedef struct student {  
    int Roll;  
    char name[20];  
} Pankaj;
```

```
void main() {  
    Pankaj s1, s2;  
    ↓ same as  
    struct student s1, s2;  
}
```

```
struct student {
```

```
    int Roll = 1;
```

```
    char name[20] = "Pankaj" ;  
};
```

~~Invalid~~

```
struct student {
```

```
    int Roll;
```

```
    char name[20];
```

```
}
```

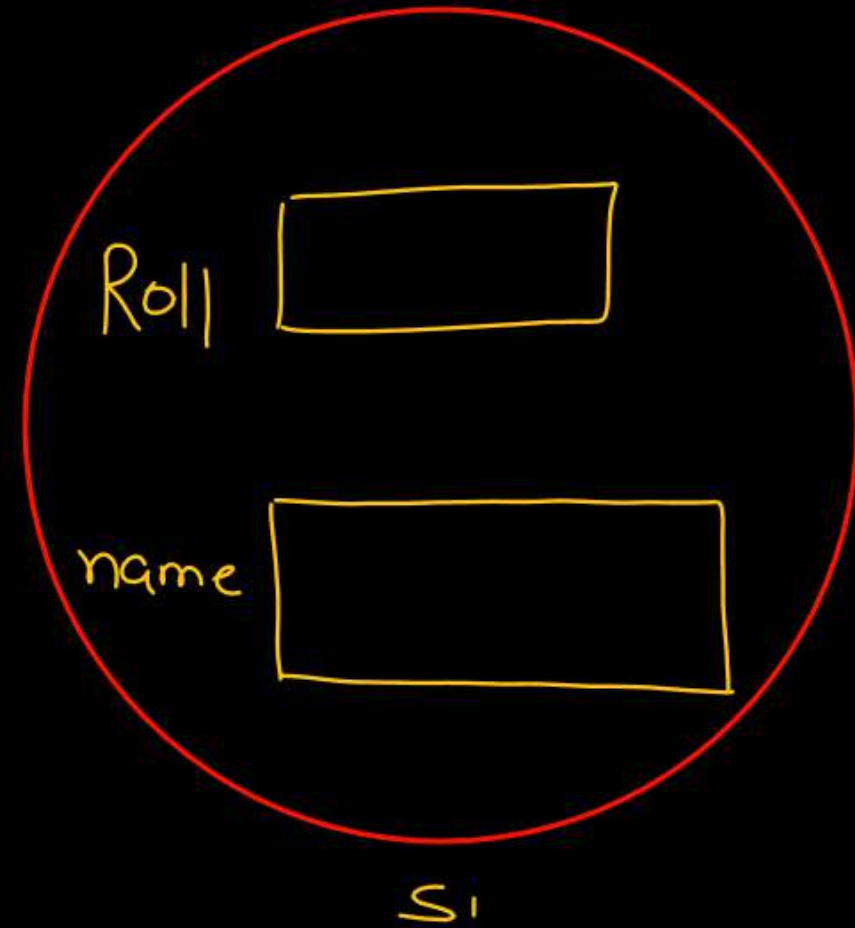
```
void main(){
```

```
    struct student s1;
```

```
    printf("/d", s1.Roll);
```

```
    printf("/s", s1.name);
```

```
}
```




```
struct student {  
    int Roll;  
    char name[20];  
};
```

Roll = "Pankaj"

```
void main() {
```

```
    struct student s = {1, "Pankaj"};
```

```
    struct student s1 = {1};
```

Invalid {

```
    struct student s3 = {"Pankaj", 10};  
    struct student s4 = {"Neeraj"};
```

Roll

1

name

Pankaj\0

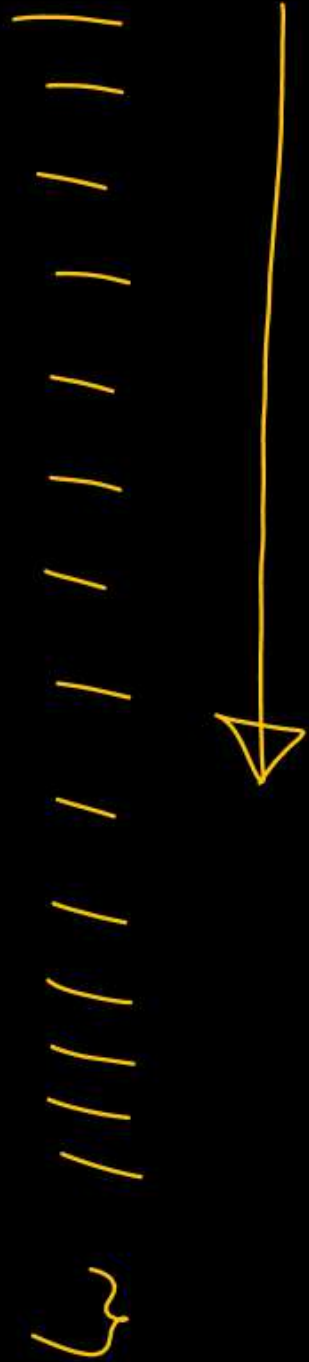
Roll

1

name

```
struct student {
```

SD attributes



99 standard $\Rightarrow$ des

⇒ designated initialization

```
struct student {  
    int Roll;  
    char name[20];  
    float marks;  
};
```

```
void main() {
```

```
    struct student s1 = { 10, "Pankaj", 14.5 };
```

*s1.Roll=10
s1.name="Pankaj" s1.marks=14.5;*

```
{ struct student s = {  
    name = "Pankaj",  
    Roll = 10,  
    marks = 14.8  
};
```

Moosay $\Rightarrow$

Is it possible to have
a static var. as a member
of a structure?

NO

Array Vs structure

name does not rep. any add.

`int a, b, c;` $\Rightarrow$ `int a[3];`

`struct student s1, s2, s3;` $\Rightarrow$ `struct structure s[3];` $\Rightarrow$ Array of structure



`s[0]  $\Rightarrow$`

`s[1]  $\Rightarrow$`

`s[2]  $\Rightarrow$`

```
struct student { int Roll;  
                 char name[20];
```

```
void display(struct student t)
```

```
{
```

```
    printf("/d", s.Roll); X
```

```
    printf("/d", t.Roll); ✓
```

```
    printf("/s", t.name); ✓
```

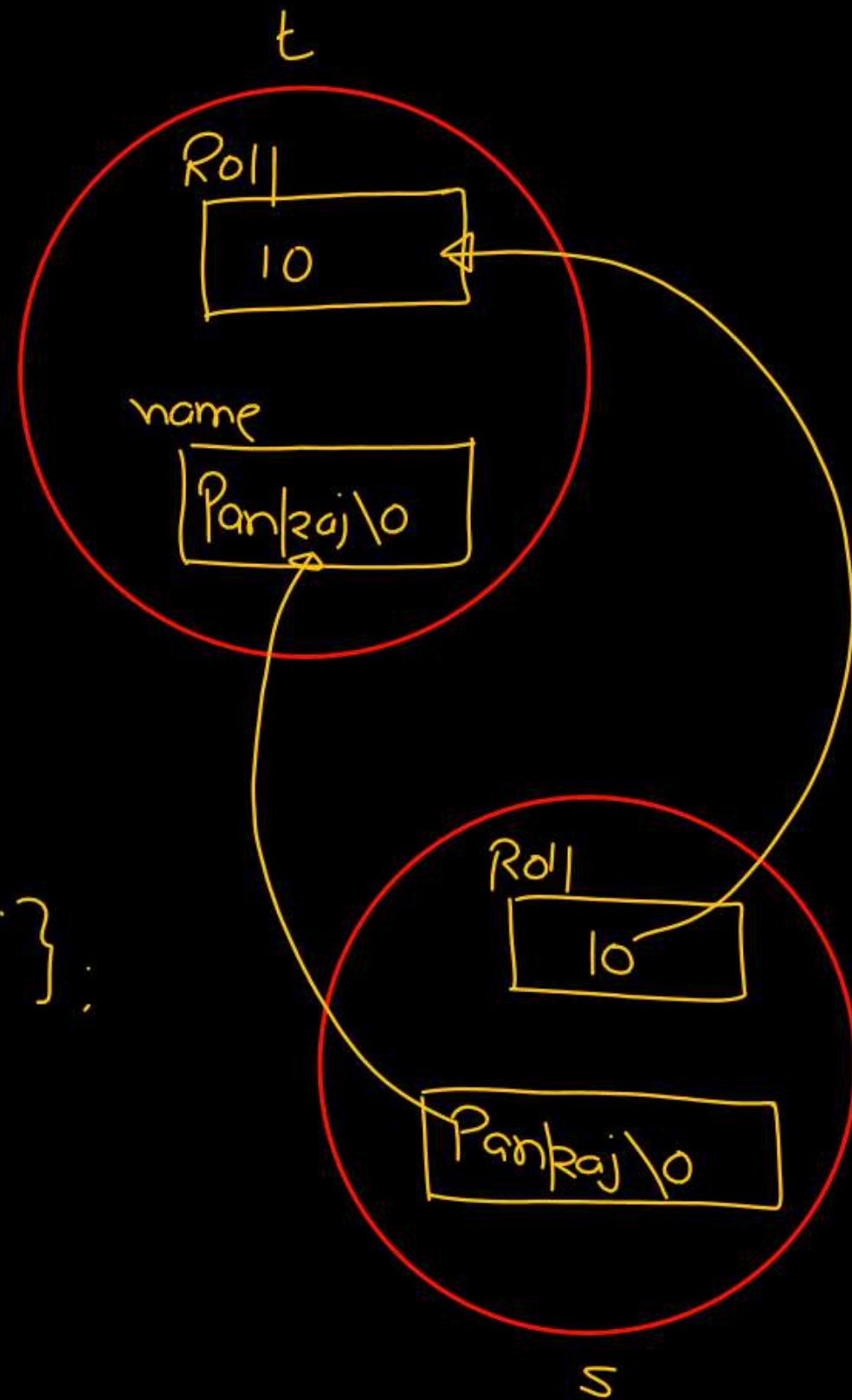
```
}
```

```
void main() {
```

```
    struct student s = {10, "Pankaj"};
```

```
    display(s);
```

```
}
```



```

struct student {
    int Roll;
    char name[20];
};

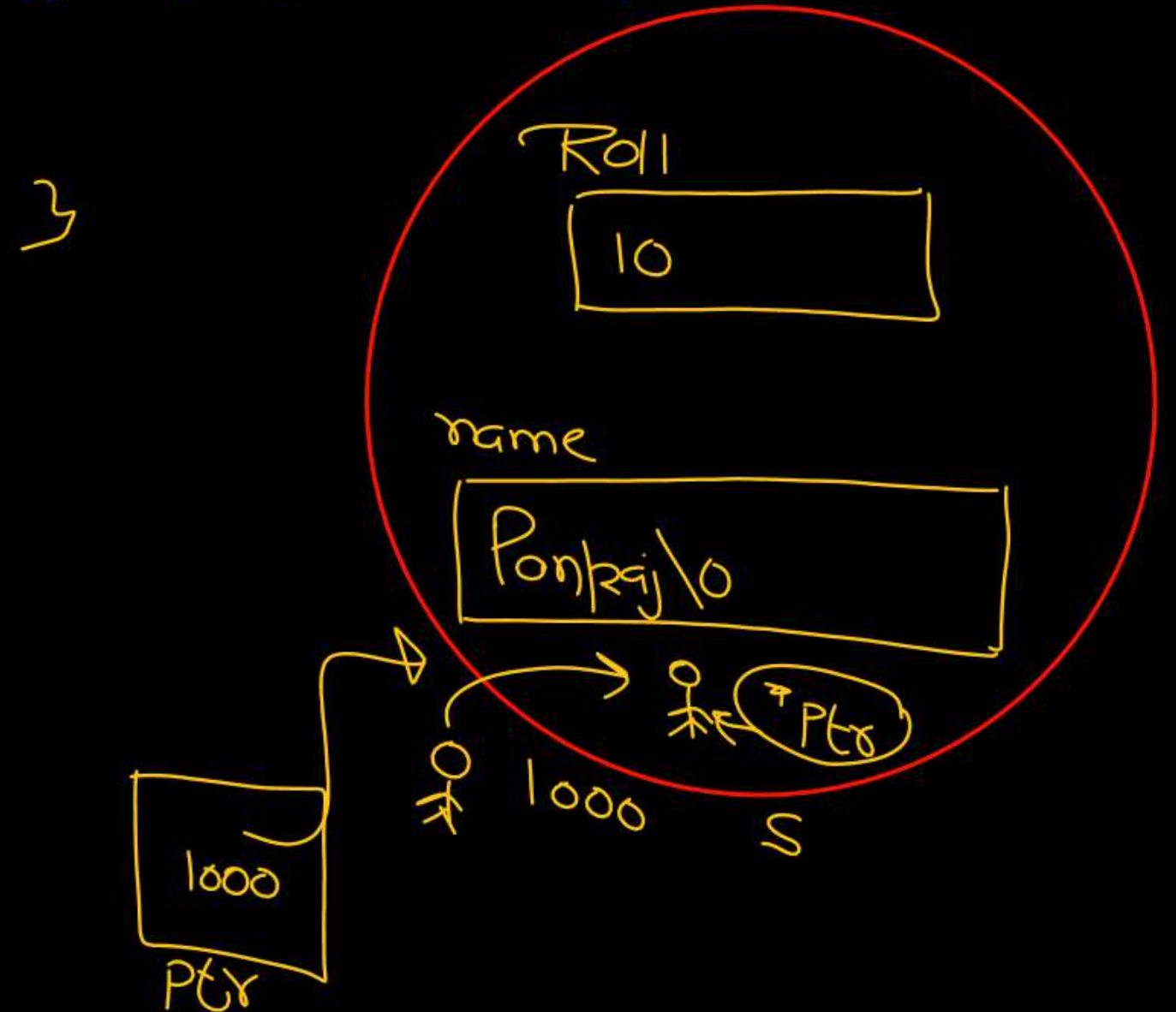
void main() {
    struct student s;
    s.Roll = 10;
    s.name = "neeraj"; X
    strcpy(s.name, "neeraj");
    display(&s);
}

```

```

void display(struct student *ptr)
{
    printf("/d", (*ptr).Roll);
    printf("/s", (*ptr).name);
}

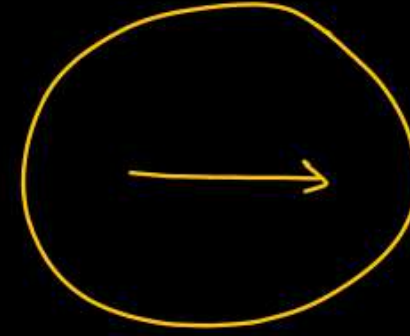
```



Ptr : Pointer to structure

(*Ptr).member1

(*Ptr).member2



Ptr -> member1 ;

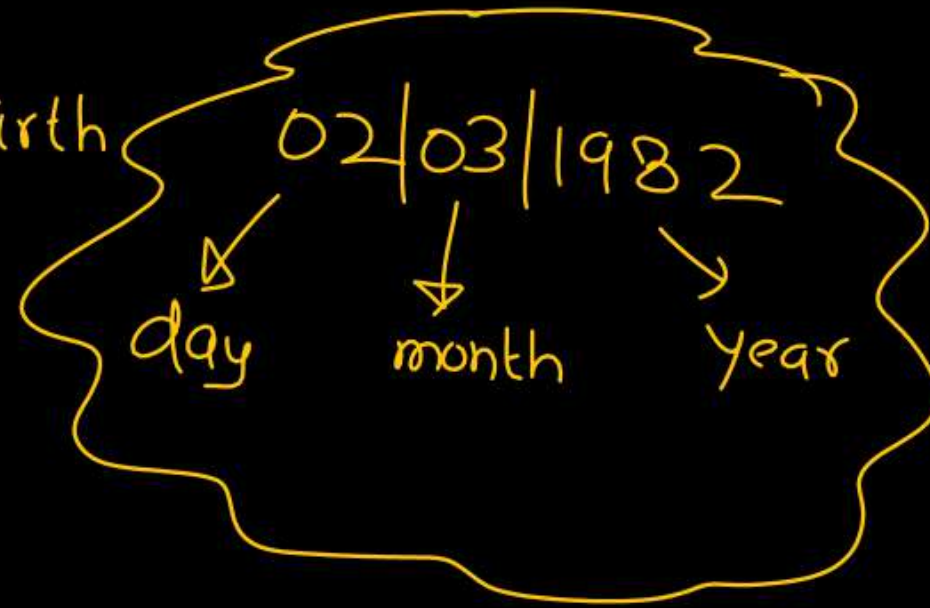
Ptr -> member2 ;



Student → Roll

→ name

→ Date of birth



```
struct DOB {
```

```
    int day;
```

```
    int month;
```

```
    int Year;
```

```
};
```

```

struct DOB {
    int day;
    int month;
    int Year;
};

```

```

struct student {
    int Roll;
    char name[20];
    struct DOB date_o_birth;
};

```

```

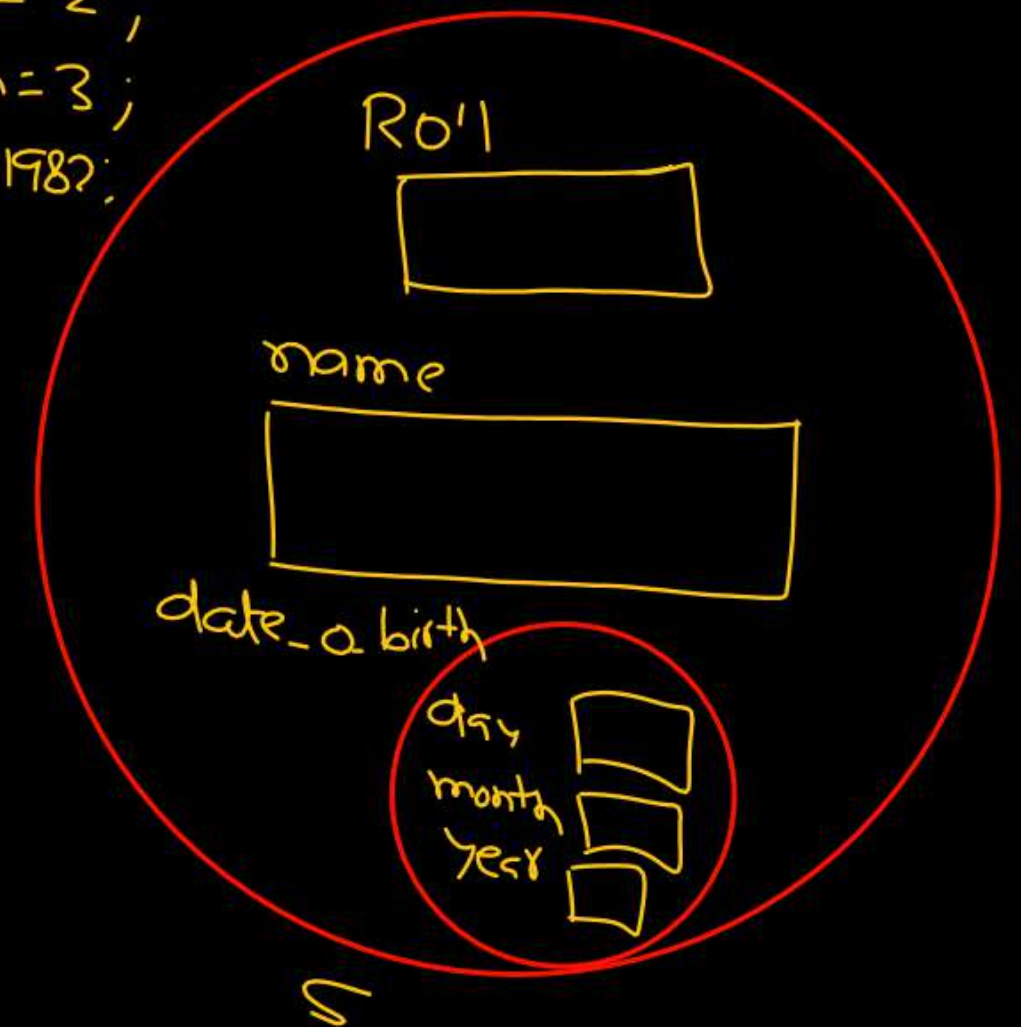
void main() {
    struct student s;
    s.Roll = 10;
    strcpy(s.name, "Pankaj");

```

```

    s.date_o_birth.day = 2;
    s.date_o_birth.month = 3;
    s.date_o_birth.Year = 1982;
}

```



```
struct student { int Roll;  
                  char name[20];  
                  struct  
                      {  
                          int day;  
                          int month;  
                          int Year;  
                          } date-of-birth;  
                  }
```

```
struct s {
```

```
    int i;
```

```
    int *a;
```

```
};
```

```
void main() {
```

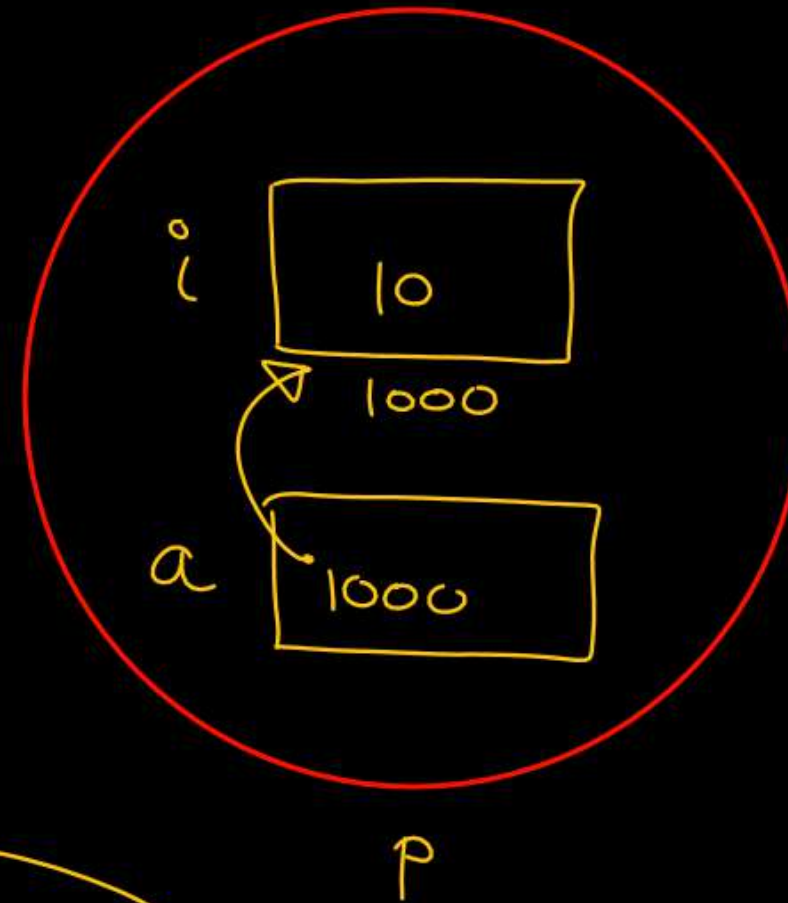
```
    struct s (p);
```

```
    p.i = 10;
```

```
    p.a =
```

```
    p.a = &p.i;
```

Add. of int. variable




```
struct s{  
    int data;
```

```
    struct s *ptr;  
};
```

```
void main(){
```

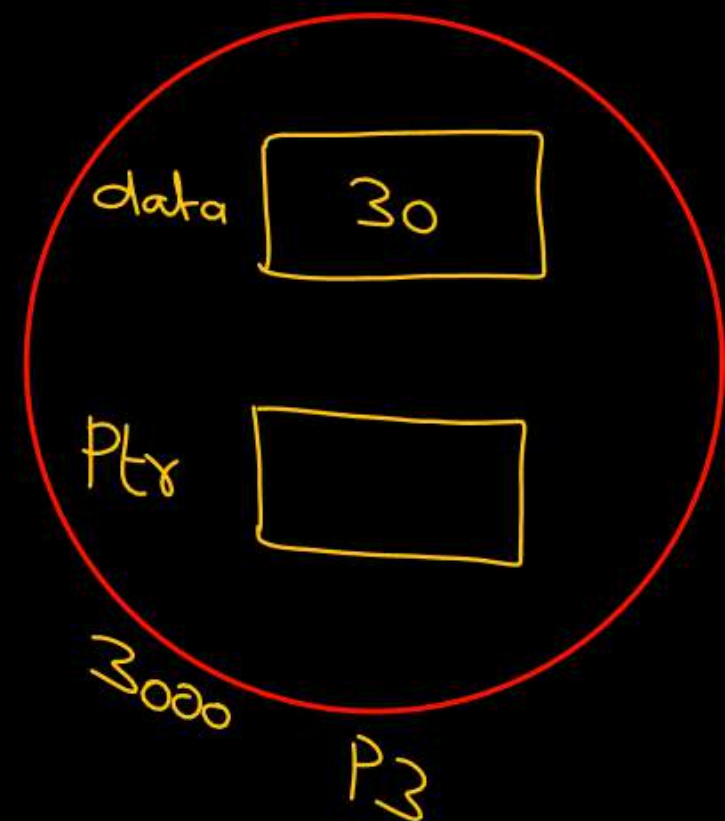
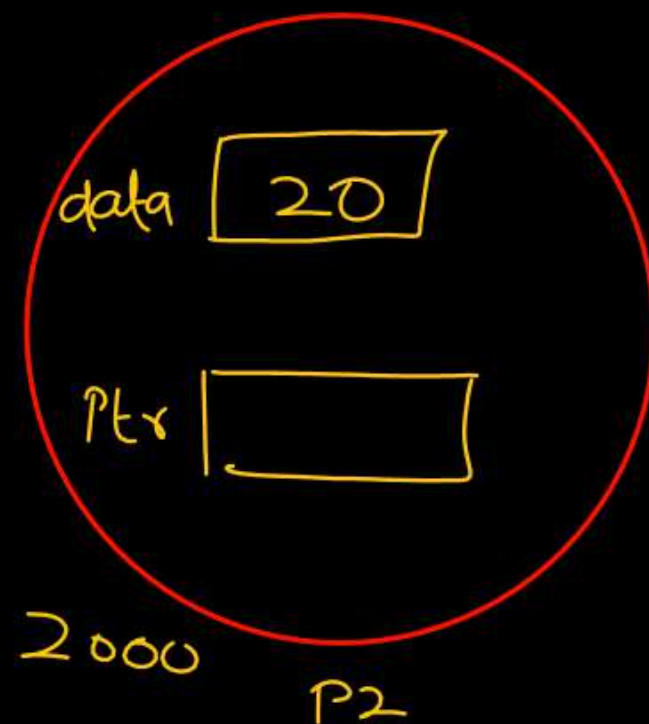
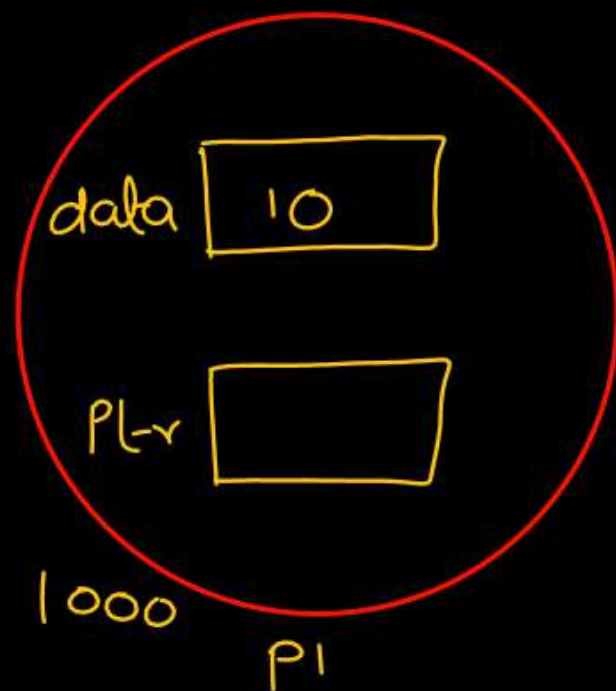
```
    struct s p1, p2, p3;
```

```
    p1.data = 10;
```

```
    p2.data = 20;
```

```
    p3.data = 30;
```

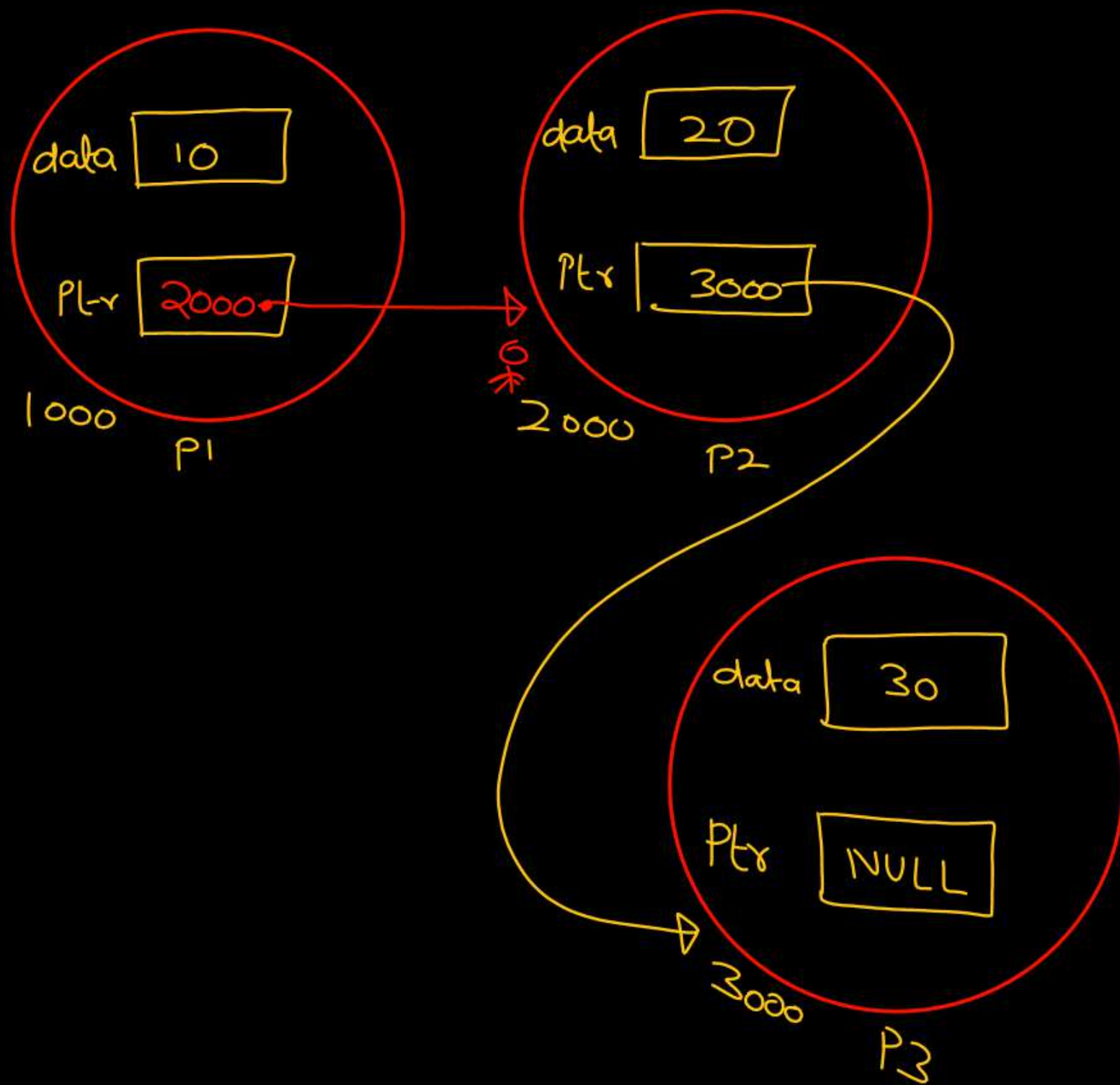
```
    p1.ptr = {Add. of var. of struct s type}
```



Self-referential structure

```
struct s{  
    int data;  
    struct s *ptr;  
};
```

```
void main(){  
    struct s p1, p2, p3;  
    p1.data = 10;  
    p2.data = 20;  
    p3.data = 30;  
    p1.ptr = &p2;  
    p2.ptr = &p3;  
    p3.ptr = NULL;  
}
```



Padding
alignment restriction } X

Last Class

- * Union
- * comma
- * sizeof
- * scoping
- * getch, getche, getchar
- * scanf — for string

